

Práctica 2: Despliegue de aplicaciones usando Kubernetes/Docker

Se pide realizar el despliegue de una aplicación remota a lo largo de un cluster de ordenadores. Se suministran los siguientes ejecutables ya compilados, el alumno debe usar estos archivos en su práctica:

- Dos programas servidor:
 - o Un programa llamado “serverFileManager”, que se corresponde con una implementación del programa servidor pedido en la práctica 2.
 - o Un programa llamado “brokerFileManager”, que se corresponde con una implementación del programa “Broker” pedido en la práctica 2.
- Un programa cliente:
 - o Un programa llamado “clienteFileManager” el cual permite conectarse a los programas servidor anteriormente mencionados y usar las funcionalidades del cliente implementado en la práctica 2.
 - o

El conjunto de programas suministrados funcionan de la siguiente manera:

- El primer programa que debe levantarse debe ser el programa “brokerFileManager”, el cual se queda a la espera de recibir conexiones del resto de aplicaciones.
- Al iniciar los programas “serverFileManager”, reciben por parámetros (argc/argv) la IP del bróker, y se conectan al programa “brokerFileManager” para darle a conocer sus datos de conexión “IP/Puerto”. A continuación se quedan a la espera de conexiones de clientes.
- La aplicación “clienteFileManager” recibe por parámetros argc/argv la IP de la máquina donde se encuentra el programa “brokerFileManager”, y se conecta para recibir datos de conexión de los programas “serverFileManager”. Cuando tiene la información de conexión, pasa a conectarse a los servidores y realizar las funcionalidades mostradas en la práctica 2.

Una vez se tienen conectados clientes con servidores, los programas “serverFileManager” se usan para permitir accesos a archivos de una carpeta local. Los programas “clienteFileManager” se conectan a estos servidores para poder listar y copiar ficheros a carpetas que se encuentran junto al programa servidor. En concreto, se disponen de los siguientes comandos desde el lado del cliente:

- **ls:**
 - o Lista los ficheros locales al programa “clientFileManager”.
- **lls:**
 - o Lista los ficheros que se encuentran en el servidor. En concreto los archivos que se encuentren en la carpeta “FileManagerDir” junto al ejecutable “serverFileManager”.
- **upload:**
 - o Este comando copia un archivo local al cliente al directorio remoto del servido. Este comando recibe un parámetro adicional: Nombre del archivo “local”, que se encuentra en el directorio local al cliente.
 - § EJ:
 - § Si se ejecuta el siguiente comando, se copiará el archivo “a.txt” del cliente servidor remoto:

- "upload a.txt"
- **exit():**
 - o Este comando cierra la terminal y acaba la ejecución del programa.
- **download:**
 - o Este comando copia un archivo remoto en el servidor al directorio local al cliente. Este comando recibe un parámetro adicional: Nombre del archivo "remoto", que se encuentra en el directorio del servidor.
 - § EJ:
 - § Si se ejecuta el siguiente comando, se copiará el archivo "a.txt" al directorio local al cliente.
 - "download a.txt"

En la siguiente práctica se pide implementar un despliegue de las aplicaciones descritas anteriormente, usando una red de ordenadores EC2 configurados en instancias/pods de kubernetes. El alumno deberá realizar una configuración de al menos 2 ordenadores en los que habrá un ordenador Maquina1 (control-plane) y uno o más nodos esclavos. En los esclavos estarán las aplicaciones tipo "servidor" mostradas anteriormente, instaladas en contenedores Docker y virtualizadas en despliegues de Kubernetes.

Las aplicaciones "cliente" se ejecutarán desde el lado del usuario, en el propio ordenador del alumno o en alguna de las instancias EC2 creadas. En concreto, para aprobar la práctica, la solución del alumno deberá cubrir los siguientes puntos:

- Creación de imágenes Docker
- Replicación de la imagen Docker a lo largo de un cluster
- Uso de Kubernetes para gestionar las replicaciones y balanceo de carga

Desarrollo de la práctica:

El alumno deberá desarrollar al menos una imagen docker para cada uno de los programas "servidor" (brokerFileManager y serverFileManager) proporcionados junto a la práctica. Esta imagen deberá estar configurada con las librerías y archivos, puertos de red, etc... necesarios para poder albergar estos ejecutables y permitir su conexión con los clientes:

- El programa serverFileManager usa el puerto 32001
- El programa brokerFileManager usa el puerto 32002

Configuración básica:

Se debe crear un cluster de kubernetes que contendrá al menos los siguientes nodos:

- Nodo master: contiene el control-plane de la aplicación

- Nodo bróker: contiene un pod con el deployment de la aplicación “brokerFileManager”
- Nodo esclavo: contiene un pod con el deployment de la aplicación “serverFileManager”

Se deben crear los servicios necesarios para poder conectar las aplicaciones cliente, bróker y servidor y que les permita realizar sus funcionalidades.

Configuraciones avanzadas para optar a 10 puntos en examen

Partiendo de la configuración anterior, se valorarán las siguientes implementaciones para sumar puntos y llegar al 10 en esta práctica:

- Configuración avanzada 1:
 - o Usar varios pods/réplicas en un único nodo esclavo:
 - § En este caso, el programa serverFileManager debe usar una carpeta compartida entre todos los pods para que cuando un cliente se conecte a alguno de ellos, vea siempre la misma carpeta/archivos y se guarden en un directorio “no volátil”.
 - Consejo: Si todos los pods están en el mismo nodo, entonces pueden usar una carpeta compartida con las opciones “hostPath”
- Configuración avanzada 2:
 - o Usar varios nodos “esclavos” para el deployment de “serverFileManager”.
 - § Al igual que en la configuración 1, se debe compartir los archivos que administran los servidores entre varios pods. En esta configuración ya no se puede usar “hostPath”, por lo que se debe usar una “unidad compartida en red”.
 - Consejo: Se puede montar un “disco duro NFS”, compatible con kubernetes. Se puede seguir esta guía de instalación/configuración:
 - o <https://www.jorgedelacruz.es/2017/12/26/kubernetes-volumenes-nfs/>

RECOMENDACIONES E INDICACIONES PARA LA ENTREGA

El ejercicio pedido es muy similar al realizado en la práctica 1, pero en este caso se suministran programas ya implementados. No es necesario que el alumno los modifique o sustituya por otros, se deben usar los ejecutables dados. Estos ejecutables se encuentran en el blackboard de la asignatura, dentro del archivo “P2FileManager.zip”.

Para familiarizarse con su funcionamiento, se deben probar estos programas dentro de las máquinas EC2. Debería comprobarse cómo funcionan cada uno de los programas por separado, y como se conectan entre sí para subir/bajar ficheros entre al menos dos máquinas EC2.

Debido a la naturaleza más general de esta práctica, se permite consultar documentación externa a la asignatura. En concreto, pueden ser interesantes tutoriales más complejos de Kubernetes/Docker. Podéis consultar con el profesor si no sabéis cómo avanzar con alguna parte de la práctica.

Por último, se aconseja ser crítico con las soluciones. Probad varias veces la demo de la sección “Entrega”, en la cual se pide realizar varias conexiones con los servidores y subir/bajar varios archivos.

Ante cualquier duda, poneros en contacto con el profesor para verificar cuanto antes que vuestro trabajo se corresponde con lo pedido.

Entrega:

Las prácticas se deben realizar de forma individual. Las prácticas se deben entregar antes del día 27 de Noviembre (23:59 del día 26), fecha en la que se realizará el examen de la práctica en clase.

Fichero ZIP con los archivos generados/editados por los alumnos. Cada uno de los ficheros entregados DEBERÁ contener el nombre de los integrantes del grupo escrito en comentarios de código al inicio de cada archivo. Se deja a elección del alumno la estructura de directorios que contendrá su entrega.

Nombre del fichero (OBLIGATORIO): PR2SISDIS_Alumno1_Apellido1.zip

Cualquier archivo entregado que no se adecúe a estas prescripciones podrá ser ignorado. Por favor, no entreguéis archivos con otros compresores o nomenclaturas.